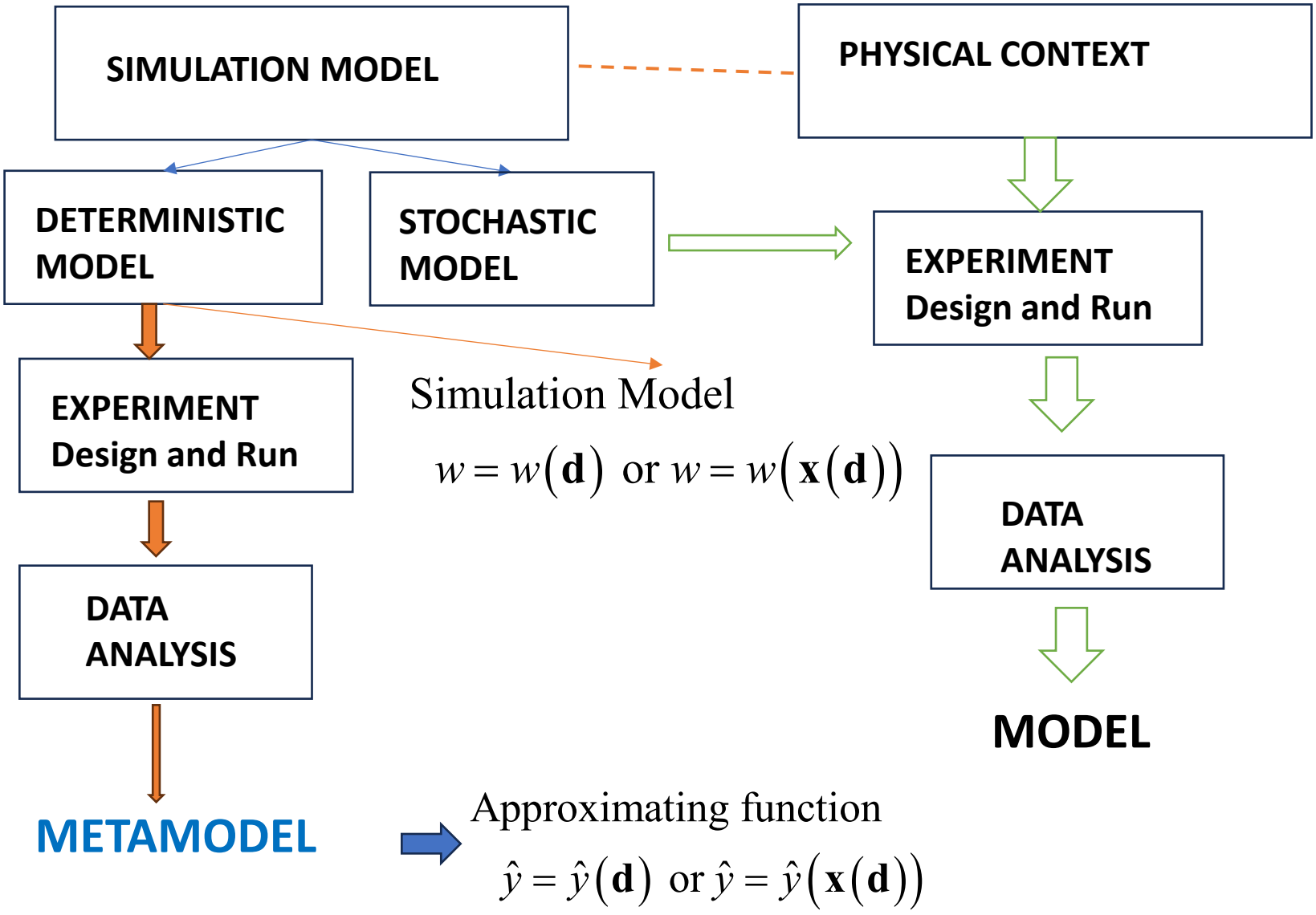
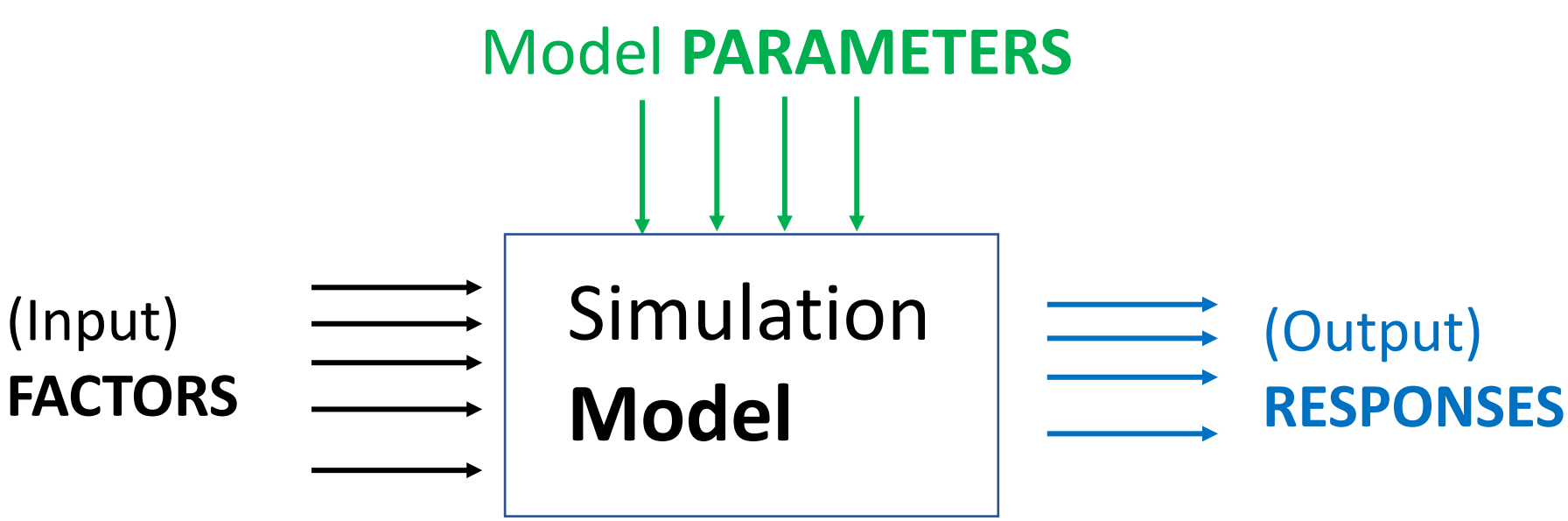
Computer Simulation Experiments

Design and Analysis


SIMULATION MODEL
PHYSICAL CONTEXT
DETERMINISTIC MODEL
STOCHASTIC MODEL
EXPERIMENT Design and Run
EXPERIMENT Design and Run
Simulation Model
$w = w(\mathbf{d})$ or $w = w(\mathbf{x}(\mathbf{d}))$
DATA ANALYSIS
DATA ANALYSIS
MODEL
METAMODEL
Approximating function
$\hat{y} = \hat{y}(\mathbf{d})$ or $\hat{y} = \hat{y}(\mathbf{x}(\mathbf{d}))$



NB: We will consider **one** response

Model variables

- ✓Control Variables (**Factors**: the experimenter assigns values)
- ✓Environmental Variables (usually they are described by probability density functions). We will not consider them.
- ✓Model Parameters (**Simulation Model Parameters** that must be calibrated)

Approaches

- **Static approach**:

we want to build a **Response function** over the complete **Experimental Region** to predict the response.

- Possible objectives:
  - ✓ **Predict** the response over the complete Experimental Region
  - ✓ **Sensitivity** analysis (**what-if** analysis)
  - ✓ **Optimization**

- **Evolutive Approach**:

we are interested in finding a **combination of factors** which guarantees something (minimum, maximum, region with response values within a constraint, etc.). The approach is **evolutive**. This is similar to **RSM**.

Work Flow (for Static Approach)

- ✓**Build** the **Simulation Model** (the model must describe the relevant aspects of your context; **Model Validation**)
- ✓**Calibrate** the Simulation Model. Sometimes it is not necessary.
- ✓**Design** the Computer Experiment (Space filling Methods)
- ✓**Run** the Experiment and collect the results

$$\text{Data set} \dashrightarrow w_i = f(\mathbf{d}_i) \quad i = 1, N$$

- ✓**Build** the **Metamodel** g (Gaussian-Kriging Process)

$$\hat{y}_i = \hat{g}\left(\mathbf{d}_i\right)$$

- ✓**Use** the Metamodel (for **prediction** or **sensitivity analysis** or **optimization**)

Analysis of Computer experiments

- The fundamental point is that we are considering **deterministic** simulation results.
- We can not use the standard linear model

$$w = f(\mathbf{d}) + \boldsymbol{\varepsilon}$$

  - We use a model **without** the random component

$$w = f(\mathbf{d})$$

- The problem turns into an **Interpolation-Extrapolation** problem
- However classical interpolation methods **do not give** the prediction uncertainty!

Example

Let us suppose to have a **single factor** d that belongs to [0,1]

You make two Computer Experiments, 5 runs each.

The results (d_i, w_i) are:

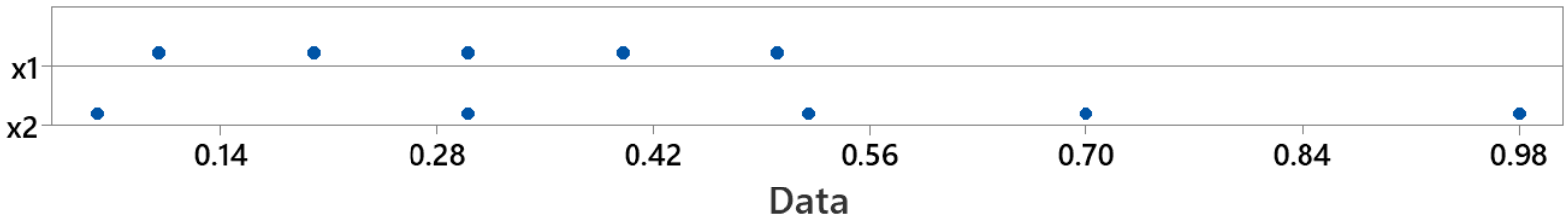
First Experiment

{(**0.1**,-0.38),(**0.2**,-0.44),(**0.3**,1.25),(**0.4**,2.11),(**0.5**,2.50)}

Second Experiment

{(**0.06**,0.62),(**0.3**,1.25),(**0.52**,2.62),(**0.69**,2.91),(**0.97**,8.29)}

Let us plot the factor values



please comment

Designs for Deterministic Simulations

When you have to build a **Metamodel** to predict the response over the entire **Design Space**, you have to use **Space-Filling Designs**.

- **Random Design:** Bad choice
- **Factorial plan:** Not recommended choice

The most popular designs are:

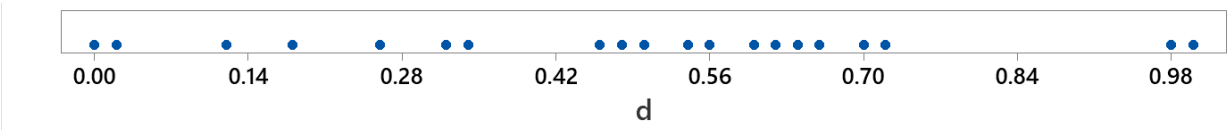
- **Latin Hypercube Design- LHD (**or **Latin Hypercube Sampling LHS)**
  - Latin Hypercube Design (LHD)
  - Orthogonal Array-based Latin Hypercube Design (OA-LHD)
- **Designs Based on Measure of Distance**
  - miniMax Distance Design (mM Design)
  - Maximin Distance Design (Mm Design)
- **Maximin Latin Hypercube Designs (MmLHD)**
- **Maximum Projection Design (MaxPro)**

There are other Space-Filling schemes (i.e. Bayesian scheme).

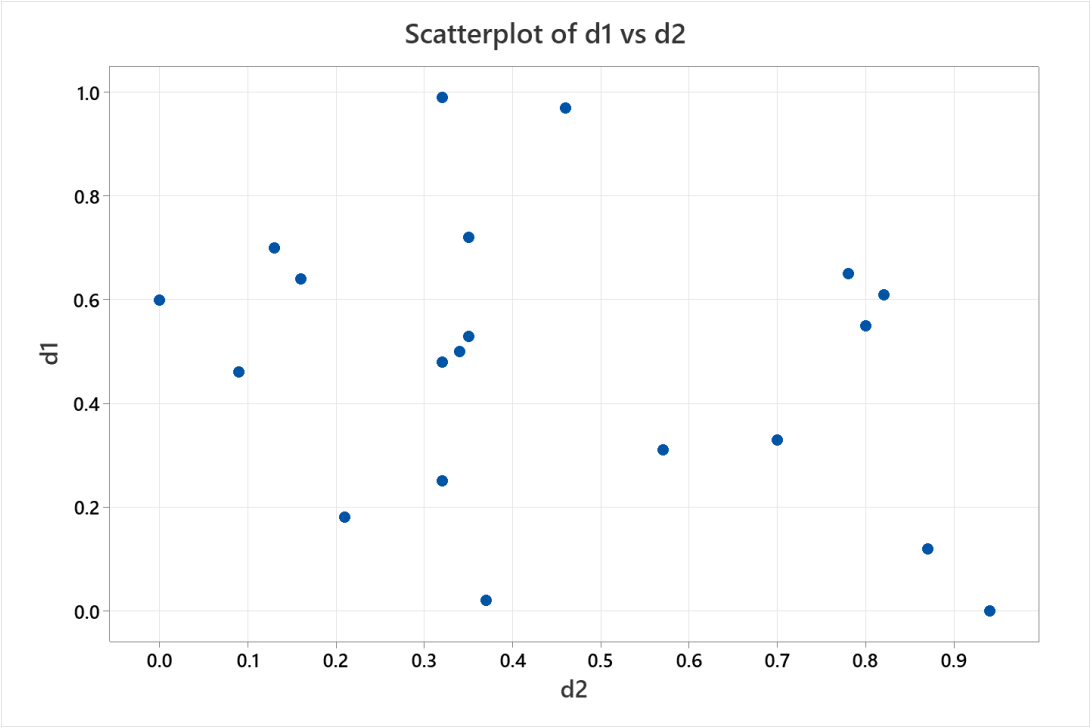
In the following we will assume that the factors range in the cube $[0,1]^q$ unless some constraints act.

Random Sample Design

- Originally proposd for the computation of the definite integral of a **many–variable** function (John von Neumann and Stanislaw Ulam) .
- *d* with one dimension (single factor) N=20

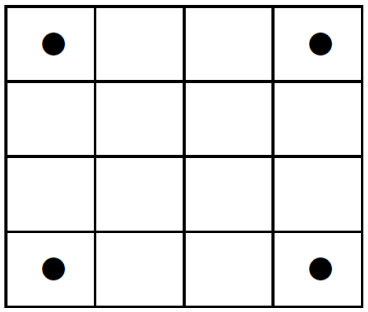


- **d** with two dimensions (two factors) N=20
  - clustered points,
  - the space is not filled regularly
- **Bad choice**



Factorial Designs

- A factorial 2^2 Design is constructed as:



$$D = \begin{bmatrix} 1 & 1 \\ 1 & 4 \\ 4 & 1 \\ 4 & 4 \end{bmatrix}$$

- If a factor is not significant, the design collapses in two replicates which are noy useful in deterministic experiments
- Remember the «**Effect sparsity**» principle: only a few factors are expected to be important

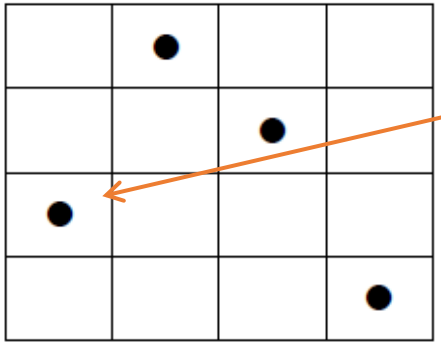
LHD Latin Hypercube Design

A **Latin Hypercube Design** (LHD) with **N** runs and **q** factors, denoted by LHD(N; q), is an N by q matrix, in which each column is a random permutation of {1, 2; … , N} .

Example with two factors **N=4 q=2**

Each **column** and **row** has one and only one treatment

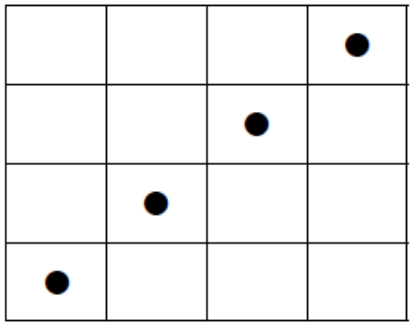
Each factor has **N** levels



$$D = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 3 \\ 4 & 1 \end{bmatrix}$$

The total number of LHD is $n_{TOT} = (N!)^q$

However: not all plans are good. For example:



$$D = \begin{bmatrix} 1 & 1 \\ 2 & 2 \\ 3 & 3 \\ 4 & 4 \end{bmatrix}$$

Note:

a) this particular design is **not** space filling

b) the factors are correlated (big problems with the $\mathbf{X}$ Matrix)

N=5 rescaling and perturbation

L =

I1	I2	I3
2	0	-2
1	-2	-1
-2	2	0
0	-1	2
-1	1	1

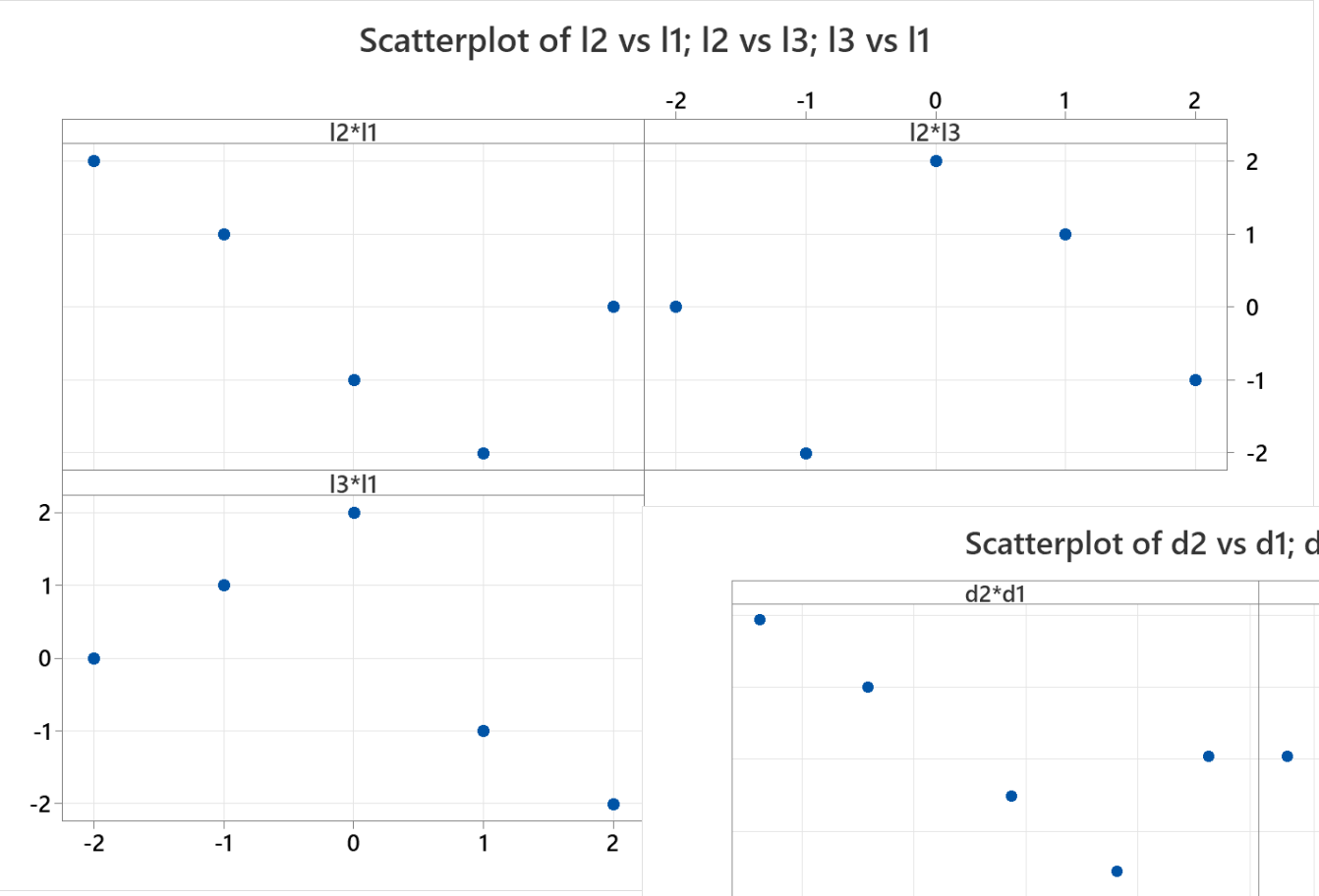
To rescale and perturbate

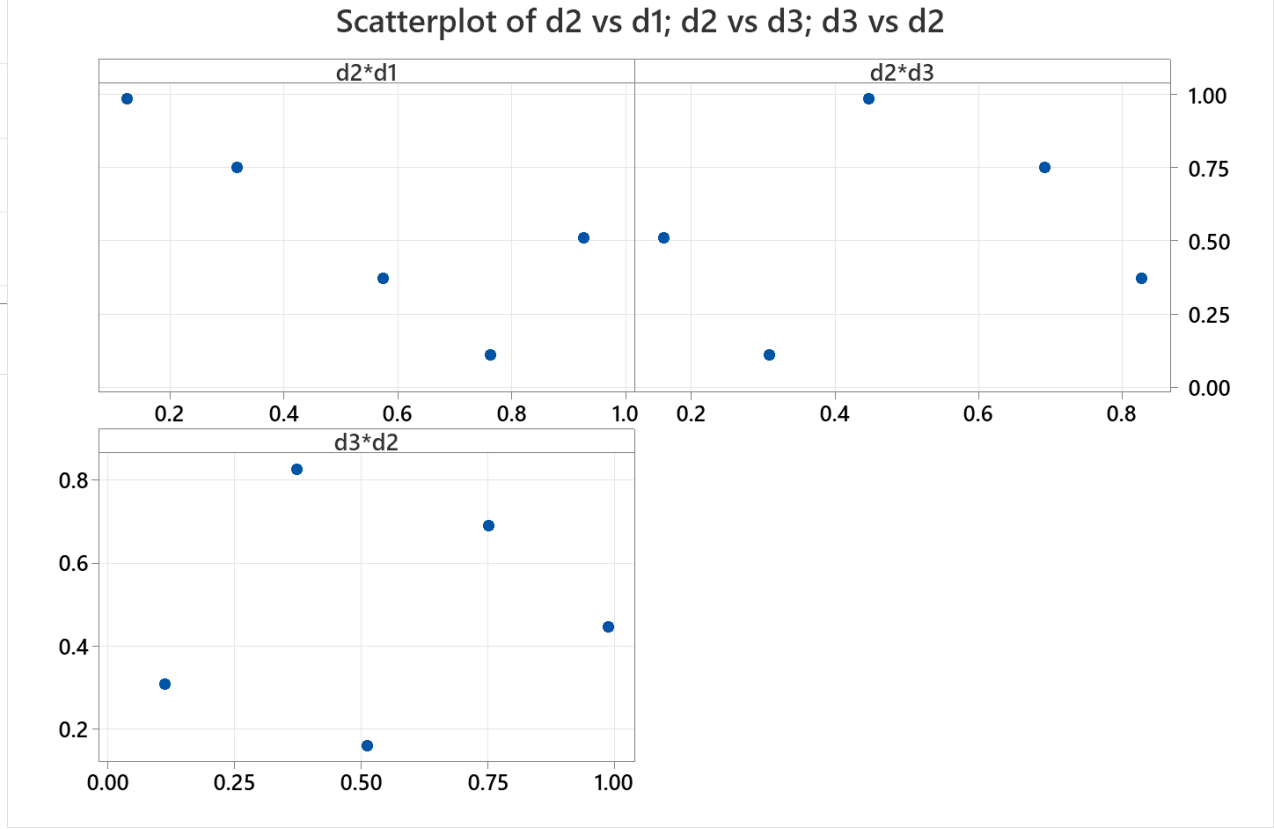
$$d_{ij} = \frac{l_{ij} + (N-1)/2 + u_{ij}}{N} \qquad \text{where } u_{ij} \sim U(0,1)$$

D =

d1	d2	d3
0.9253	0.5117	0.1610
0.7621	0.1117	0.3081
0.1241	0.9878	0.4473
0.5744	0.3719	0.8270
0.3181	0.7514	0.6916

If u_{ij}=0.5 -> D is called Lattice Sample


Scatterplot of l2 vs l1; l2 vs l3; l3 vs l1
l2*l1
l2*l3
l3*l1
-2
-1
0
1
2

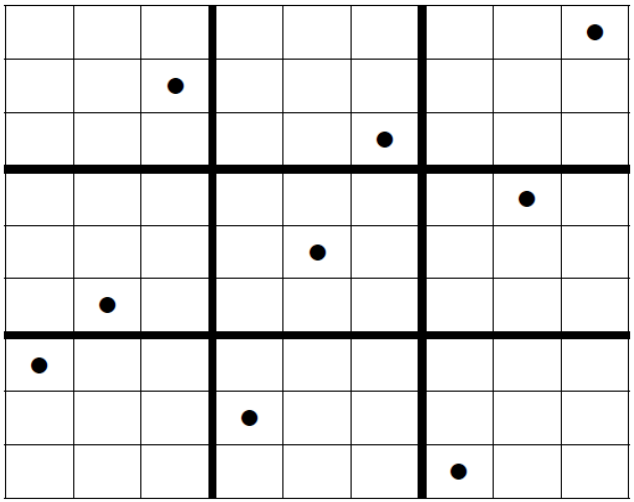

Scatterplot of d2 vs d1; d2 vs d3; d3 vs d2
d2*d1
d2*d3
d3*d2
0.2
0.4
0.6
0.8
1.0
0.00
0.25
0.50
0.75
1.00

Orthogonal Array-based Latin Hypercube Design (OA-LHD)

Example: **N=9** and **q=2**

$$OA \rightarrow \begin{bmatrix} d_1 & d_2 \\ 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 2 & 1 \\ 2 & 2 \\ 2 & 3 \\ 3 & 1 \\ 3 & 2 \\ 3 & 3 \end{bmatrix}$$

$$1 \rightarrow \{1,2,3\} \rightarrow \{3,2,1\}$$
$$2 \rightarrow \{4,5,6\} \rightarrow \{4,5,6\}$$
$$3 \rightarrow \{7,8,9\} \rightarrow \{8,7,9\}$$



$$OA - LHD \rightarrow \begin{bmatrix} d_1 & d_2 \\ 1 & 3 \\ 2 & 4 \\ 3 & 8 \\ 4 & 2 \\ 5 & 5 \\ 6 & 7 \\ 7 & 1 \\ 8 & 6 \\ 9 & 9 \end{bmatrix}$$

Design Based on Measures of Distance

Let us define a metric between two points $\mathbf{d_1}$ and $\mathbf{d_2}$ $\quad \rho(\bullet,\bullet)$

Metric Properties:

$$\rho(\mathbf{d}_1,\mathbf{d}_2) \geq 0$$

$$\rho(\mathbf{d}_1,\mathbf{d}_2) = 0 \quad \text{if } \mathbf{d}_1 = \mathbf{d}_2$$

$$\rho(\mathbf{d}_1,\mathbf{d}_2) = \rho(\mathbf{d}_2,\mathbf{d}_1)$$

$$\rho(\mathbf{d}_1,\mathbf{d}_2) \leq \rho(\mathbf{d}_1,\mathbf{d}_3) + \rho(\mathbf{d}_3,\mathbf{d}_2)$$

For example: $$\rho(\mathbf{u},\mathbf{v}) = \left\{ \sum_{j=1,q} \left| u_j - v_j \right|^k \right\}^{1/k} = \left(\left\| \mathbf{u} - \mathbf{v} \right\|_k \right)^{1/k}$$

k=1 -> Rectangular Distance

k=2 -> Euclidean Distance

miniMax Distance Design (mM)

Experimental Region of **q factors** $\chi_D = [0,1]^q$

Design $D = (\mathbf{d}_1, \mathbf{d}_2, \ldots \mathbf{d}_N) \qquad \mathbf{d}_i \in \chi_D$

minimum distance for any point $\mathbf{d}$ with $\mathbf{d} \in \chi_D \rightarrow \rho(\mathbf{d}, D) = \min_{\mathbf{d}_i \in D} \rho(\mathbf{d}, \mathbf{d}_i)$

Fill distance $h(D) = \max_{\mathbf{d} \in \chi_D} \rho(\mathbf{d}, D)$

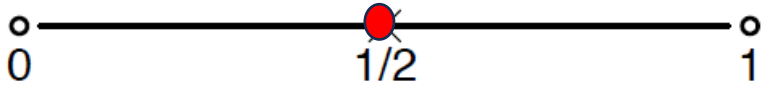
h is the radius of the largest ball that can be placed in χ_D which **does not contain any point** in the design D

miniMax distance Design (mM): Find a design D to minimize h

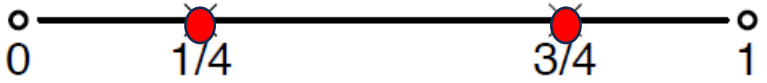
$$\min_D h(D) \quad \Rightarrow \quad \min_D \max_{\mathbf{d} \in \chi_D} \rho(\mathbf{d}, D)$$

Examples

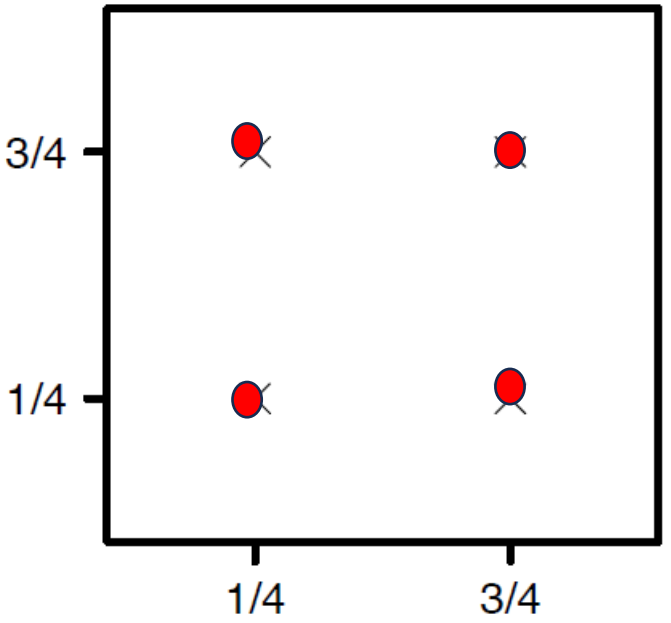
q =1 N=1



q =1 N=2



q =2 N=4



Consider the owner of a petroleum corporation who wants to open some gas stations. mM design ensures that **no customer is too far f**rom one of the company's gas stations.

Maximin Distance Design (Mm Design)

The minimum distance between **any two** points in *D* is

$$2s = \min_{\mathbf{d}_i, \mathbf{d}_j \in D} \rho\left(\mathbf{d}_i, \mathbf{d}_j\right)$$

where s is the **separation distance** or **packing radius** - the radius of the largest ball that can be placed **around** every design point such that **no ball overlaps with another**.

Find *D* that maximizes $2s$

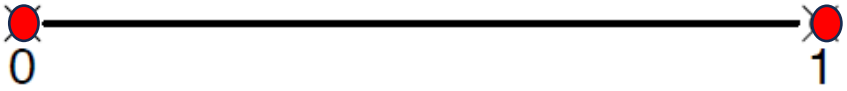
$$\max_{D} \min_{\mathbf{d}_i, \mathbf{d}_j \in D} \rho\left(\mathbf{d}_i, \mathbf{d}_j\right) \Rightarrow$$ Maximin distance design (Mm)

A large s ensures numerical stability in Kriging.

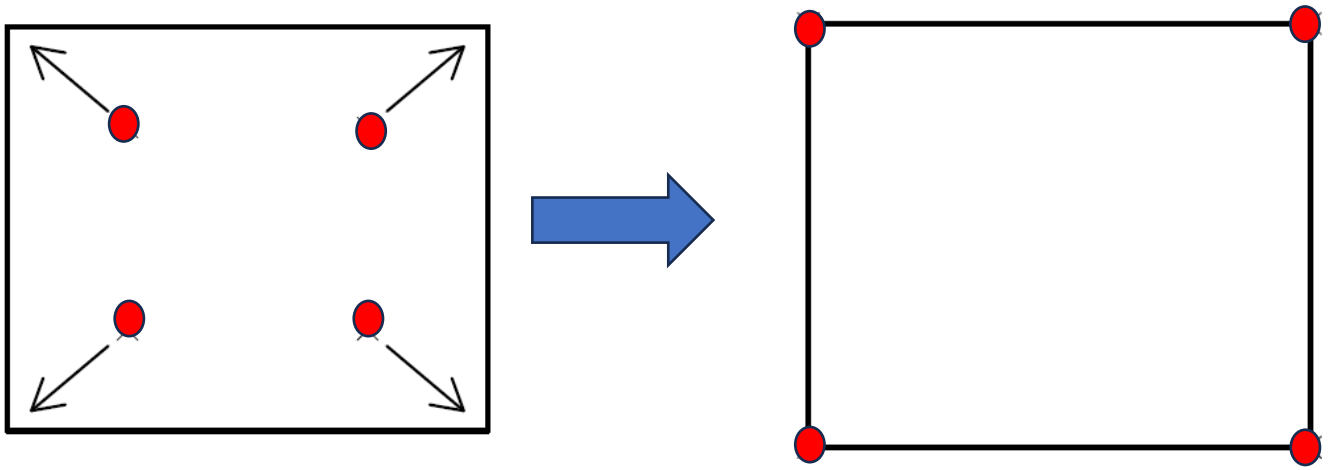
A large s tends to decrease h.

Examples

q =1 N=2



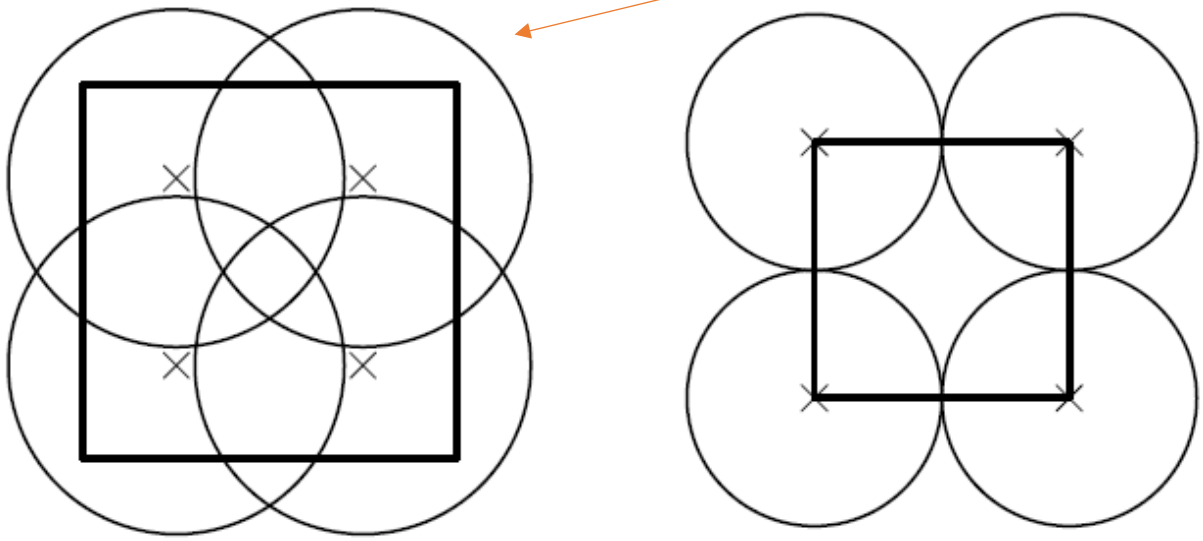
q =2 N=4



Gas station example: Mm design ensures that **no two gas stations** are too close to each other. It minimizes the competition from each other by locating the stations as far apart as possible.

In mathematics:

- **Covering** problems: cover $\mathbb{R}^q$ with hyperspheres ->**miniMax**
- **Packing** problems: pack hyperspheres in $\mathbb{R}^q$ ->**Maximin**

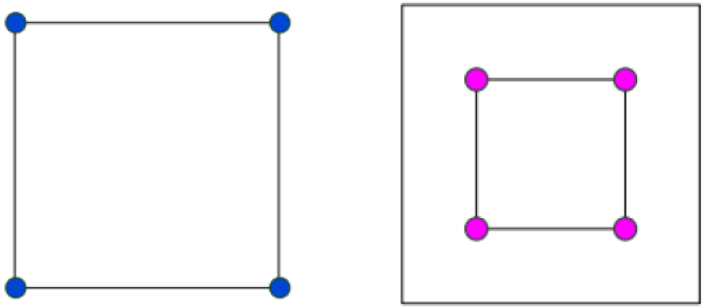


(a) Minimum radius. (b) Maximum radius.

The two problems are very hard to solve (Packing less than Covering)

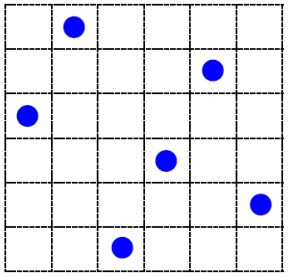
i.e. R package: `minimaxdesign` by Mak&Joseph 2018

Maxmin Latin Hypercube Designs (MmLHD)



(a) Maximin design (b) Minimax design.

- Advantage of mM/Mm designs: **run size flexibility** and **optimality**
- Disadvantage of mM/Mm designs: **projections** are **poor**



LHD: good 1-dimensional projections, but can be poor in terms of **Space-filling** in higher dimensions.

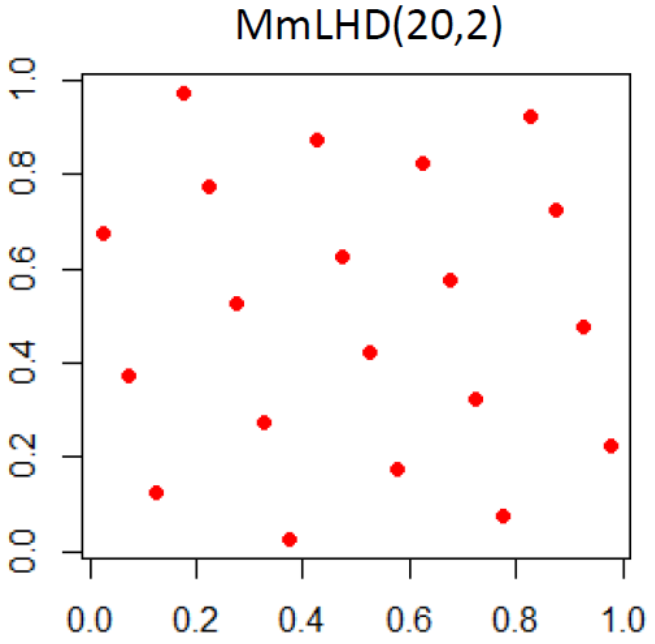
Combine these two ideas:

Maximin Latin Hypercubes Designs (MnLHD)

$$\max_{D} \min_{\mathbf{d}_i, \mathbf{d}_j \in D} \rho\left(\mathbf{d}_i, \mathbf{d}_j\right) \quad \text{where } D \text{ is an LHD}$$

This is equivalent to:

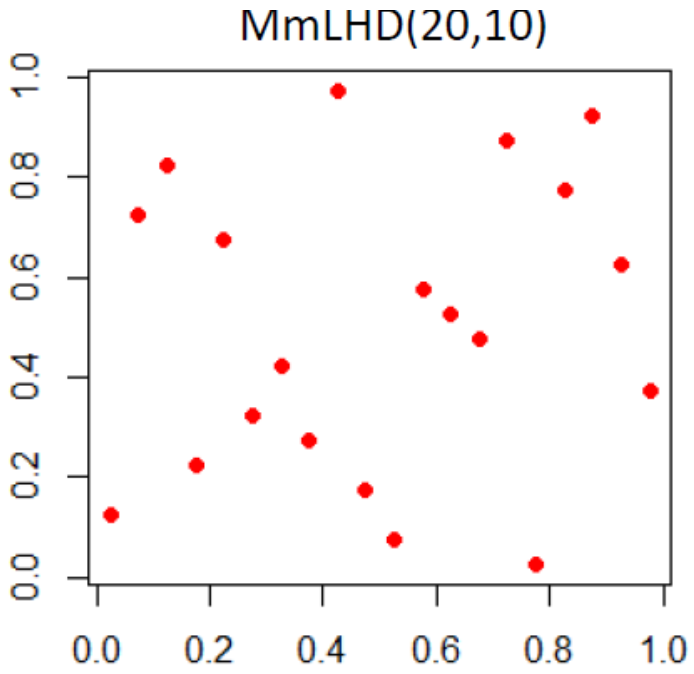
$$\min_{D} \left(\sum_{i=1}^{N-1} \sum_{j=i+1}^{N} \frac{1}{\rho\left(\mathbf{d}_i, \mathbf{d}_j\right)^k} \right)^{1/k} \quad \text{with } k \to \infty, \text{ and } D \text{ is LHD}$$



Maximum Projection Design (MaxPro)

MmLHDs ensure good space-fillingness in q dimensions and uniform projections in a single dimension, but projection properties in 2, 3, . . . q -1 dimensions **may not be good**.

In practice, we often have **1** < number of imp. Factors < **q**.



Projection in two dimensions for a 10 dimensional MmLHD.

How to calculate distances in a **projected subspace**?

Let $0 \le \theta_l \le 1$ be the weight assigned to the factor l and $\sum_{l=1}^{q} \theta_l = 1$

Weighted Euclidean Distance $$\delta\left(\mathbf{d}_i, \mathbf{d}_j; \boldsymbol{\theta}\right) = \left\{ \sum_{l=1}^{q} \theta_l \left(d_{il} - d_{jl} \right)^2 \right\}^{1/2}$$

The MmLHD criterion is modified to:

$$\min_{D} \phi_k \left(D; \boldsymbol{\theta} \right) = \left(\sum_{i=1}^{N-1} \sum_{j=i+1}^{N} \frac{1}{\delta\left(d_i, d_k; \boldsymbol{\theta} \right)^k} \right)^{1/k}$$

$$\boldsymbol{\theta} = \left(\theta_1, \theta_2 \ldots, \theta_{q-1} \right)^T \quad \text{and} \quad \theta_q = 1 - \sum_{l=1}^{q-1} \theta_l$$

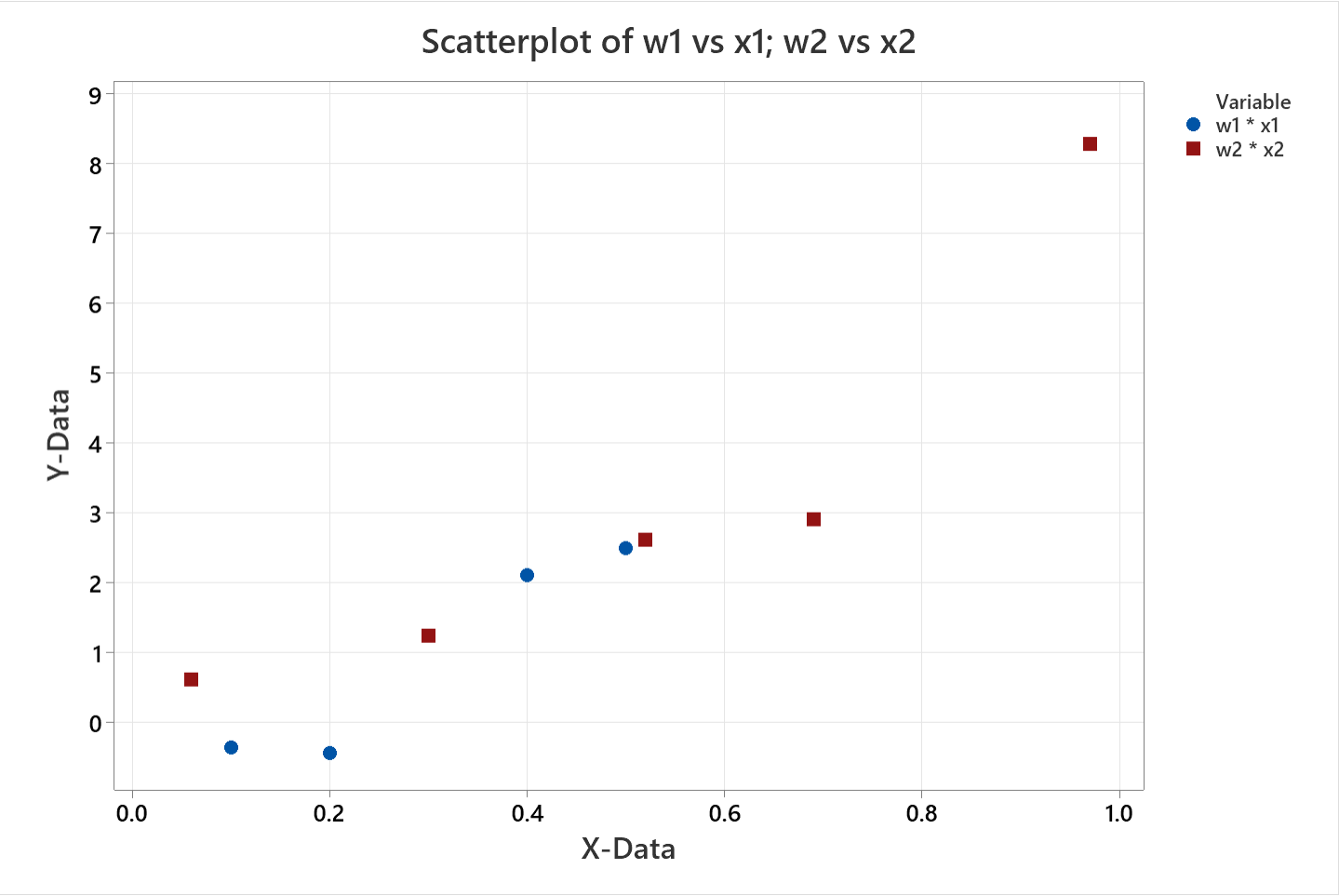
The problem is that we have no idea about the importance of the factors before the experiment.

We solve the problem with a **Bayesian approach** and maximize Space-filling properties on projections to all possible subsets (available in JMP and in R -> `MaxPro`)

Conclusions

- **Latin hypercube design:**
  - Good 1-dimensional projections, but might not be space-filling in q dimensions.
- **miniMax/Maximin distance design:**
  - Run size flexibility and good space-fillingness in q dimensions, but **bad projections**.
- **Maximin Latin Hypercube design:**
  - Good space-fillingness in q dimensions and uniform projections in a single dimension.
- **Maximum projection design:**
  - Good space-filling properties in projections to all subsets of factors, but it is difficult to define the weight vector

Let us go back to the previous simple experiments

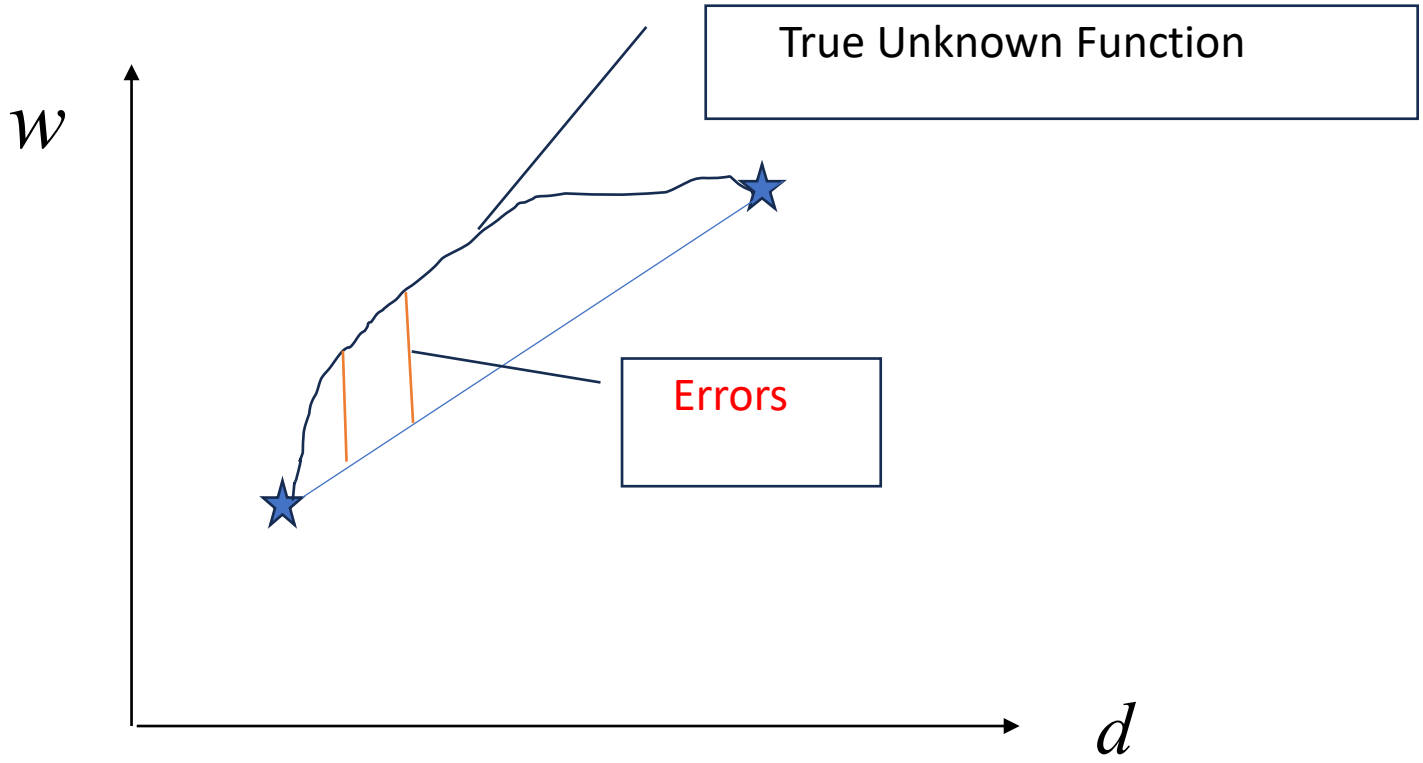


Questions:

- May we use the data of the **first** experiment to predict for d= 1.2?
- How good is the interpolation for d= 1.2 for the **second** experiment?

Gaussian Process (Kriging) Model

- The fundamental idea is to build a **Stochastic Model** using deterministic data: it seems a contradiction, but it works!



- The errors are **Spatially Correlated** (positive correlation)
- We will use a **Stationary Gaussian Process**

Stationary Gaussian Process

- In the following we will consider a **single factor** but we will generalize to a vector (a treatment with more than one factor).
- Let us consider the process $\left\{M(d), d \in \chi_D\right\}$
- This is a **Gaussian Process** (**GP**) if:

$\forall$ finite set $\left\{d_1, d_2, ..., d_m\right\} \rightarrow \left\{M(d_1), M(d_2), ..., M(d_m)\right\} \sim N_m(\boldsymbol{\mu}, \boldsymbol{\Sigma})$

moreover

$$\boldsymbol{\mu}(d) = E\left(M(d)\right) \quad \boldsymbol{\sigma}^2(d) = Var\left(M(d)\right) \quad \boldsymbol{\sigma}\left(d_i, d_j\right) = \mathrm{cov}\left(M(d_i), M(d_j)\right)$$

- The GP is **stationary** if

$$\boldsymbol{\mu}(d) = \mathrm{cost} = 0 \quad \boldsymbol{\sigma}^2(d) = \mathrm{cost} \quad \boldsymbol{\sigma}\left(d_i, d_j\right) = \mathrm{function}\left(\left\|d_i - d_j\right\|_2\right)$$

$$\text{Euclidean distance } \left\|d_i - d_j\right\|_2 \Rightarrow \sqrt{\left(d_i - d_j\right)^2} \Rightarrow \left|d_i - d_j\right|$$

The **UK-Universal Kriging** (Full Model) is

$$W(d) = \mathbf{x}(d)^T \boldsymbol{\beta} + M(d) \Rightarrow \text{Training set} \quad \mathbf{W}_D = \mathbf{X}\boldsymbol{\beta} + \mathbf{M}_D$$

$d \rightarrow$ single factor in which we want to predict

it could be generalized to $\mathbf{d} \rightarrow$ (q-dimensional vector)

$D = \{d_1, d_2, ...d_N\} \rightarrow$ Design used to have the training set

$\mathbf{x}(d) \rightarrow$ vector of regressors $\{x_0(d), x_1(d), ..., x_k(d)\}$

$\boldsymbol{\sigma}^2 \rightarrow$ "Scale factor" or "Process Variance"

$R_{ij} \rightarrow$ Spatial Correlation Function between i and j

$$\boldsymbol{\sigma}^2 R_{ij} = \text{cov}\left(M(d_i), M(d_j)\right) \rightarrow \boldsymbol{\sigma}^2 \mathbf{R}$$

$$\boldsymbol{\sigma}^2 \mathbf{r}(d) = \left\{\text{cov}\left(M(d), M(d_1)\right), .., \text{cov}\left(M(d), M(d_N)\right)\right\}^T$$

In the following we will discuss:

- ✓How to **predict** the **Response** and its **Variance**
- ✓How to **estimates** the **Model Parameters**
- ✓**Simplificatio**n from **Universal Kriging** to **Ordinary Kriging**
- ✓**Generalization** to **Multi Factor** designs: from d_i to $\mathbf{d}_i$

Let us consider:

a) Design $\boldsymbol{D}$ (single factor) $D = \{d_1, d_2 \ldots d_N\}$

b) the **Response vector (Training set)**.

$$\mathbf{w}_D = \{w_1, w_2 \ldots w_N\}^T$$

The random counterpart of $\mathbf{w}_D$ is $\mathbf{W}_D = \{W_1, W_2, \ldots W_N\}^T$

Formulas to predict the response and its variance

- We want to estimate the response in a **generic point** d belonging to the Experimental Region χ_D
- We want a **BLUP** (**B**est **L**inear **U**nbiased **P**redictor)

$$\text{Linear Predictor in } d \rightarrow \hat{y}(d) = \mathbf{c}^T \mathbf{W}_D$$

The predictor must be **unbiased**:

$$E\left(\hat{y}(d)\right) = E\left(W(d)\right)$$

Unbiasedness condition (see Supplemental material, **point 2**)

$$\boxed{\mathbf{X}^T \mathbf{c} = \mathbf{x}(d)}$$

Note: we have to find the vector of costants $\mathbf{c}$

$$MSPE\left[\hat{y}(d)\right] = E\left(\left(\hat{y}(d) - W(d)\right)^2\right) = E\left[\left(\mathbf{c}^T\mathbf{W}_D - W(d)\right)^2\right]$$

After some algebraic manipulation (see Supplemental material, **point 3**)

$$MSPE\left[\hat{y}(d)\right] = \sigma^2\left(1 + \mathbf{c}^T\mathbf{R}\mathbf{c} - 2\mathbf{c}^T\mathbf{r}(d)\right)$$

The optimization problem becomes:

$$\min_{\mathbf{c}} MSPE\left[\hat{y}(d)\right] = \sigma^2\left(1 + \mathbf{c}^T\mathbf{R}\mathbf{c} - 2\mathbf{c}^T\mathbf{r}(d)\right)$$

$$\text{s.t. } \mathbf{X}^T\mathbf{c} = \mathbf{x}(d)$$

The solutions (see Supplemental material, **point 4**) are:

$$\hat{y}(d) = \mathbf{x}^T(d)\boldsymbol{\beta}^* + \mathbf{r}(d)^T \mathbf{R}^{-1}\left(\mathbf{w}_D - \mathbf{X}\boldsymbol{\beta}^*\right)$$

$$\boldsymbol{\beta}^* = \left(\mathbf{X}^T\mathbf{R}^{-1}\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{R}^{-1}\mathbf{w}_D$$

Note:

$\mathbf{x}^T(d)\boldsymbol{\beta}^* = \boldsymbol{\beta}^{*T}\mathbf{x}(d) \rightarrow$ Trend estimate

$\mathbf{r}^T(d)\mathbf{R}^{-1}\left(\mathbf{w}_D - \mathbf{X}\boldsymbol{\beta}^*\right) \rightarrow$ GP (local) contribution

if d is very far from $\{d_1, d_2, ..., d_N\} \rightarrow \mathbf{r}(d) = \mathbf{0}$

if $\mathbf{R} = \mathbf{I} \Rightarrow \boldsymbol{\beta}^* \rightarrow \hat{\boldsymbol{\beta}} = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{w}_D$ (OLS)

The GP contribution is an interpolation of the residuals

of the regression model $\mathbf{w}_D - \mathbf{X}\boldsymbol{\beta}^* = \mathbf{m}_D$

The **Variance** of the **Predictor** (see Supplemental material, **point 5**) is:

$$MSPE = \sigma^2 \left(1 - \mathbf{r}(d)^T \mathbf{R}^{-1} \mathbf{r}(d) + \mathbf{u}^T \left(\mathbf{X}^T \mathbf{R}^{-1} \mathbf{X}\right)^{-1} \mathbf{u}\right)$$

Where $\mathbf{u}$ (dimension: p x1) is: $\mathbf{u}(d) = \mathbf{X}^T \mathbf{R}^{-1} \mathbf{r}(d) - \mathbf{x}(d)$

Or with a **less precise** formula:

$$V\left[\hat{y}(d)\right] = \sigma^2 \left(1 - \mathbf{r}(d)^T \mathbf{R}^{-1} \mathbf{r}(d)\right)$$

NOTE:

1. We can use these formulas if **we know** $\mathbf{r}$ and $\mathbf{R}$
2. We need to estimate the process variance

The next step is to estimate $\mathbf{r}$, $\mathbf{R}$ and σ^2

$$\sigma^2 R_{ij} = \mathrm{cov}\left(M\left(d_i\right), M\left(d_j\right)\right) \rightarrow \sigma^2 \mathbf{R}$$

$$\boldsymbol{\sigma}^2 r\left(d_i\right) = \mathrm{cov}\left(M\left(d\right), M\left(d_i\right)\right) \rightarrow \boldsymbol{\sigma}^2 \mathbf{r}\left(d\right)$$

The distance (Euclidean) between two treatments is:

$$\delta = \left|d_i - d_j\right| \quad \text{or if we have a vector } \mathbf{d} \quad \delta = \left\|\mathbf{d}_i - \mathbf{d}_j\right\|_2$$

A tipical expression for **correlation** (weights) is:

p = 1 -> Exponential

p = 2 -> Gaussian

$$R_{ij}\left(\boldsymbol{\delta} \mid \boldsymbol{\theta}, p\right) = \exp\left[-\boldsymbol{\theta}\boldsymbol{\delta}^p\right]$$

Other functions have been proposed: **Linear**, **Spherical**, **Cubic**, etc.

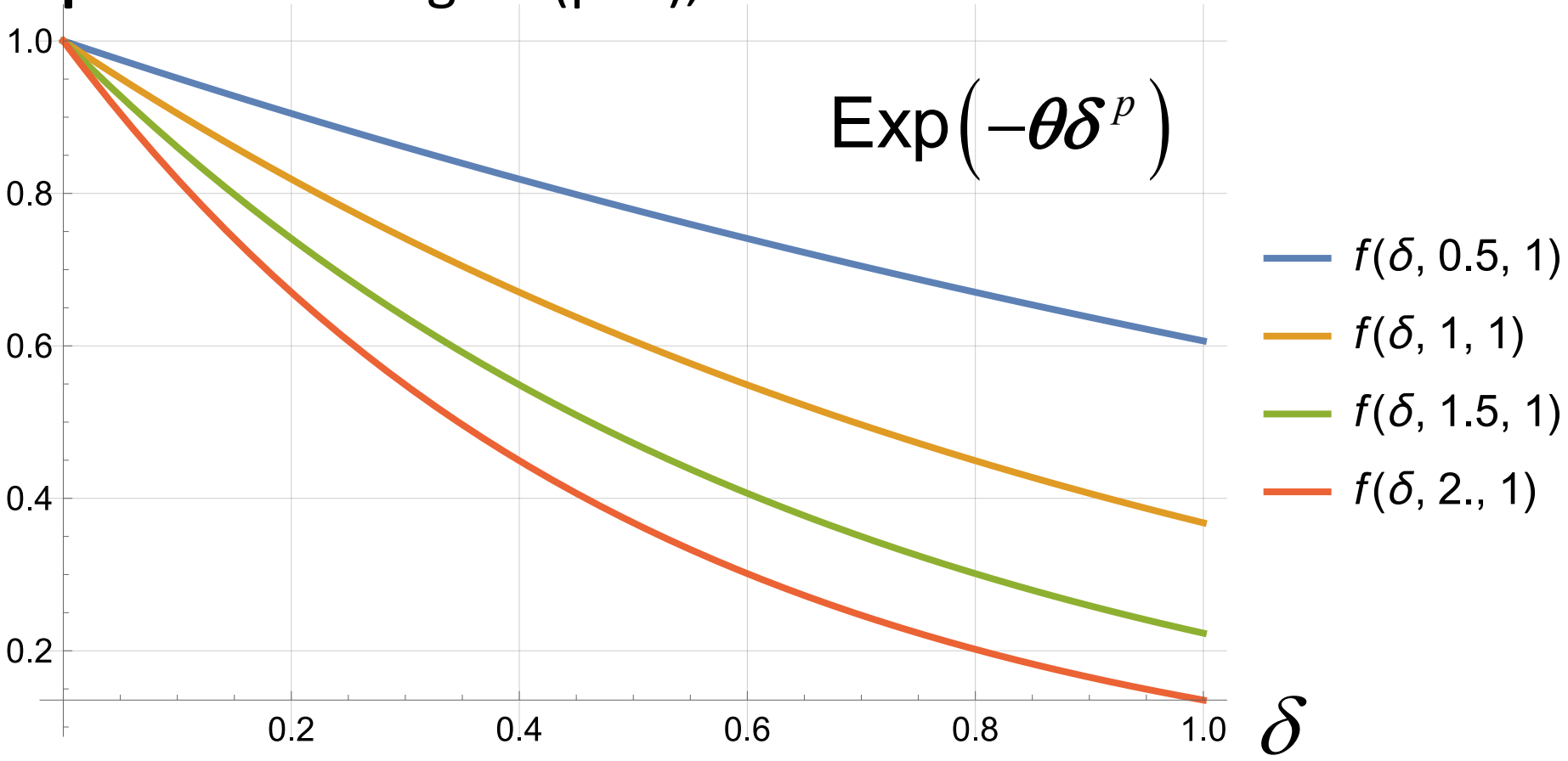
We will use **Gaussian weights**

Weight properties:

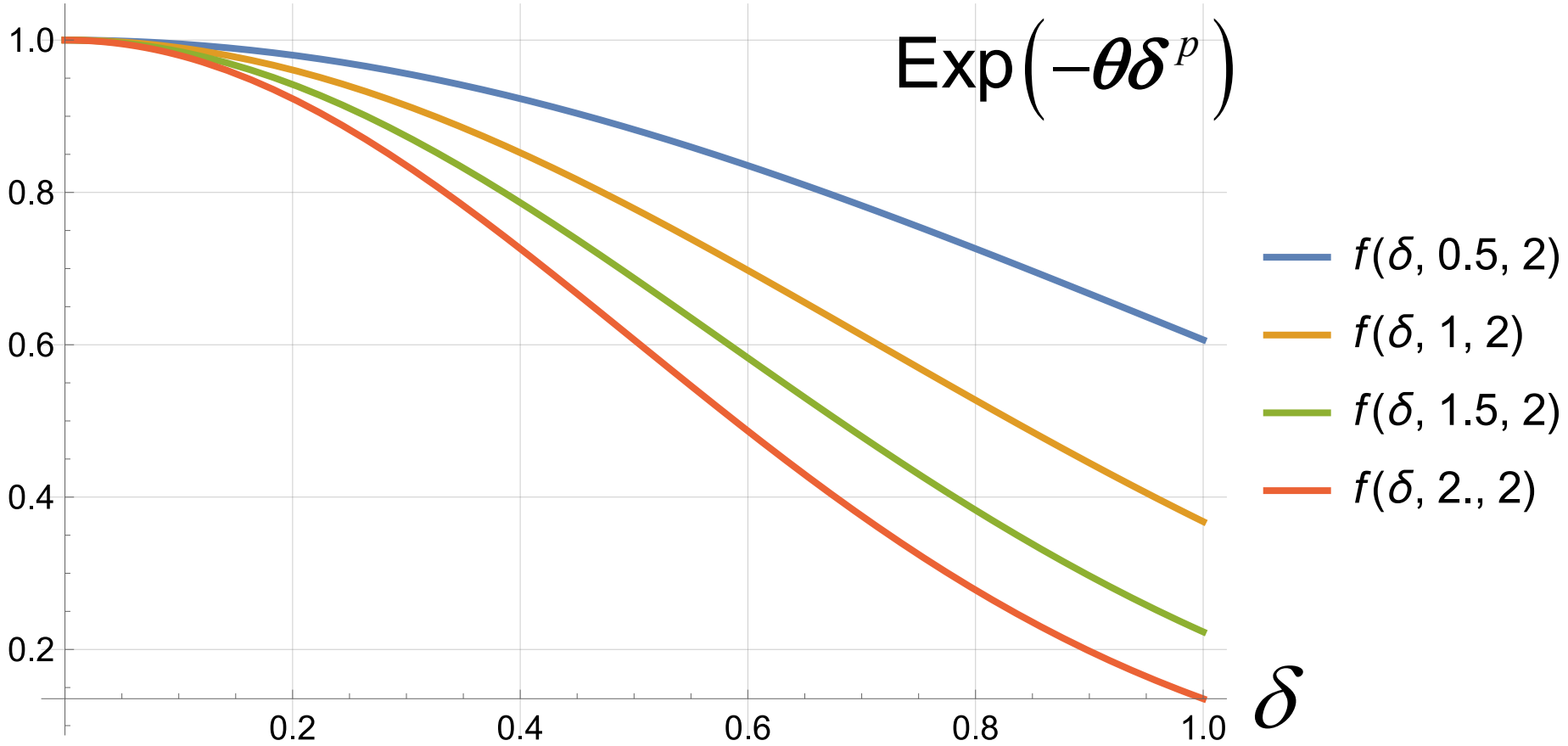
$\text{if } \delta \to \infty \Rightarrow R_{ij} = 0$ very distant treatment has no influence

$\text{if } \delta = 0 \Rightarrow R_{ij} = 1$ Perfectly correlated with itself

Exponential Weights (p=1), Theta from 0.5 to 2

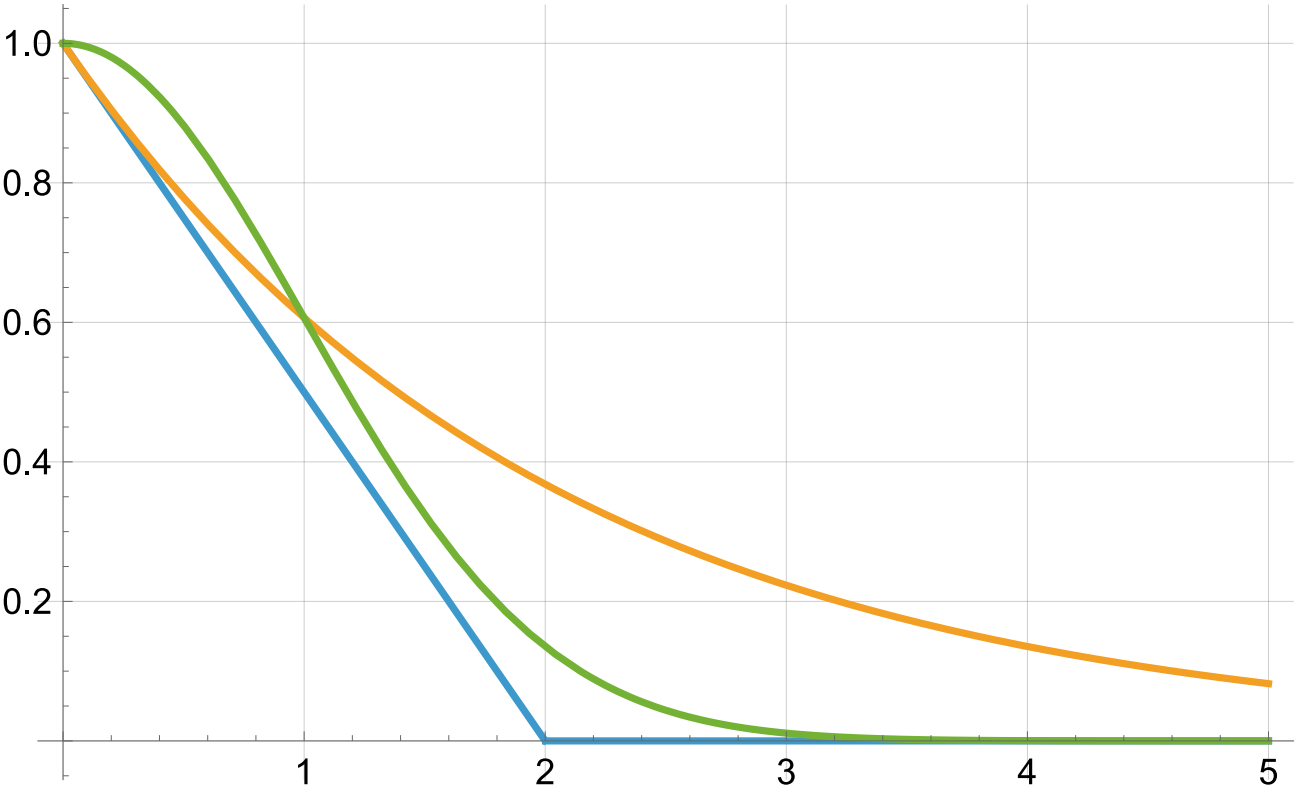


Gaussian Weights (p=2) Theta from 0.5 to 2



Linear weights

$$R_{ij}\left(\delta \mid \theta\right) = \max\left(1 - \theta\delta, 0\right)$$



Very large

MLE Maximum Likelihood Estimator

$$\mathbf{W}_D : \{W(d_1), W(d_2), ..., W(d_N)\} \sim N_N\left(\mathbf{X}\boldsymbol{\beta}, \sigma^2 \mathbf{R}(\theta, p)\right)$$

We have to estimate: $\boldsymbol{\beta}$, σ, θ e p The **LogLikelihood** function is:

$$l\left(\boldsymbol{\beta}, \sigma^2, \theta, p\right) = -\frac{N}{2}\log\sigma^2 - \frac{1}{2}\log\left|\mathbf{R}(\theta, p)\right| - \frac{1}{2\sigma^2}\left(\mathbf{w}_D - \mathbf{X}\boldsymbol{\beta}\right)^T \mathbf{R}(\theta, p)\left(\mathbf{w}_D - \mathbf{X}\boldsymbol{\beta}\right)$$

The optimization technique works in **two steps** (one of several possible algorithms).

First, we find

$$\boldsymbol{\beta}^*(\theta, p) = \left(\mathbf{X}^T \mathbf{R}(\theta, p)^{-1} \mathbf{X}\right)^{-1} \mathbf{X}^T \mathbf{R}(\theta, p)^{-1} \mathbf{w}_D$$

$$\sigma^*(\theta, p) = \left(\frac{1}{N}\left(\mathbf{w}_D - \mathbf{X}\boldsymbol{\beta}^*\right)^T \mathbf{R}(\theta, p)^{-1}\left(\mathbf{w}_D - \mathbf{X}\boldsymbol{\beta}^*\right)\right)^{1/2}$$

Second, we substitute the two expressions in the Loglikelihood function and **optimize it numerically**

$$l^*(\theta, p) = -\frac{1}{2}\left(N\log\sigma^*(\theta, p)^2 + \log\left|\mathbf{R}(\theta, p)\right|\right)$$

Then we substitute the found values and we obtain the two following expressions: $\mathbf{R}\left(\hat{\boldsymbol{\theta}}, \hat{p}\right) \rightarrow \hat{\mathbf{R}} \quad \mathbf{r}\left(d \mid \hat{\boldsymbol{\theta}}, \hat{p}\right) \rightarrow \hat{\mathbf{r}}(d)$

$$\hat{\boldsymbol{\beta}}_{MLE} = \left(\mathbf{X}^T \hat{\mathbf{R}}^{-1} \mathbf{X}\right)^{-1} \mathbf{X}^T \hat{\mathbf{R}}^{-1} \mathbf{w}_D$$

$$\hat{\sigma}^2 = \frac{1}{N}\left(\mathbf{w}_D - \mathbf{X}\hat{\boldsymbol{\beta}}_{MLE}\right)^T \hat{\mathbf{R}}^{-1}\left(\mathbf{w}_D - \mathbf{X}\hat{\boldsymbol{\beta}}_{MLE}\right)$$

$$\hat{y}(d) = \mathbf{x}^T(d)\hat{\boldsymbol{\beta}}_{MLE} + \hat{\mathbf{r}}(d)^T \hat{\mathbf{R}}^{-1}\left(\mathbf{w}_D - \mathbf{X}\hat{\boldsymbol{\beta}}_{MLE}\right)$$

after setting $\mathbf{u}(d) = \mathbf{X}^T \hat{\mathbf{R}}^{-1} \hat{\mathbf{r}}(d) - \mathbf{x}(d)$

$$V\left(\hat{y}(d)\right) \equiv MSPE = \hat{\sigma}^2\left(1 - \mathbf{r}(d)^T \hat{\mathbf{R}}^{-1} \hat{\mathbf{r}}(d) + \mathbf{u}^T\left(\mathbf{X}^T \hat{\mathbf{R}}^{-1} \mathbf{X}\right)^{-1} \mathbf{u}\right)$$

Usually we set $p=2$ (Gaussian Weighting scheme) and the numerical optimization is lightened.

Simplification: Ordinary Kriging

We can apply the Kriging working scheme **without** the trend component **->** **Ordinary Kriging**

The model becomes:

$$Y(d) = \beta + M(d) \begin{cases} E[Y(d)] = \beta \;\; \text{since } E[M(d)] = 0 \\ \text{var}[Y(d)] = \text{var}[M(d)] = \sigma^2 \\ \text{cov}[Y(d_1), Y(d_2)] = \text{cov}[M(d_1), M(d_2)] \end{cases}$$

You can use the same formulas we discussed with $\mathbf{X}^T \rightarrow \mathbf{1}^T$

The formulas becomes:

$$\hat{\boldsymbol{\beta}} = \left(\mathbf{1}^T \hat{\mathbf{R}}^{-1} \mathbf{1}\right)^{-1} \mathbf{1}^T \hat{\mathbf{R}}^{-1} \mathbf{w}_D$$

$$\hat{\boldsymbol{\sigma}}^2 = \frac{1}{N}\left(\mathbf{w}_D - \mathbf{1}\hat{\boldsymbol{\beta}}\right)^T \hat{\mathbf{R}}^{-1}\left(\mathbf{w}_D - \mathbf{1}\hat{\boldsymbol{\beta}}\right)$$

$$\hat{y}(d) = \hat{\boldsymbol{\beta}} + \hat{\mathbf{r}}(d)^T \hat{\mathbf{R}}^{-1}\left(\mathbf{w}_D - \hat{\boldsymbol{\beta}}\mathbf{1}\right)$$

$$V\left(\hat{y}(d)\right) = \boldsymbol{\sigma}^2\left(1 - \hat{\mathbf{r}}(d)^T \hat{\mathbf{R}}^{-1}\mathbf{r}(d) + \frac{\left[1 - \mathbf{1}^T \hat{\mathbf{R}}^{-1}\hat{\mathbf{r}}(d)\right]^2}{\mathbf{1}^T \hat{\mathbf{R}}^{-1}\mathbf{1}}\right)$$

Note:

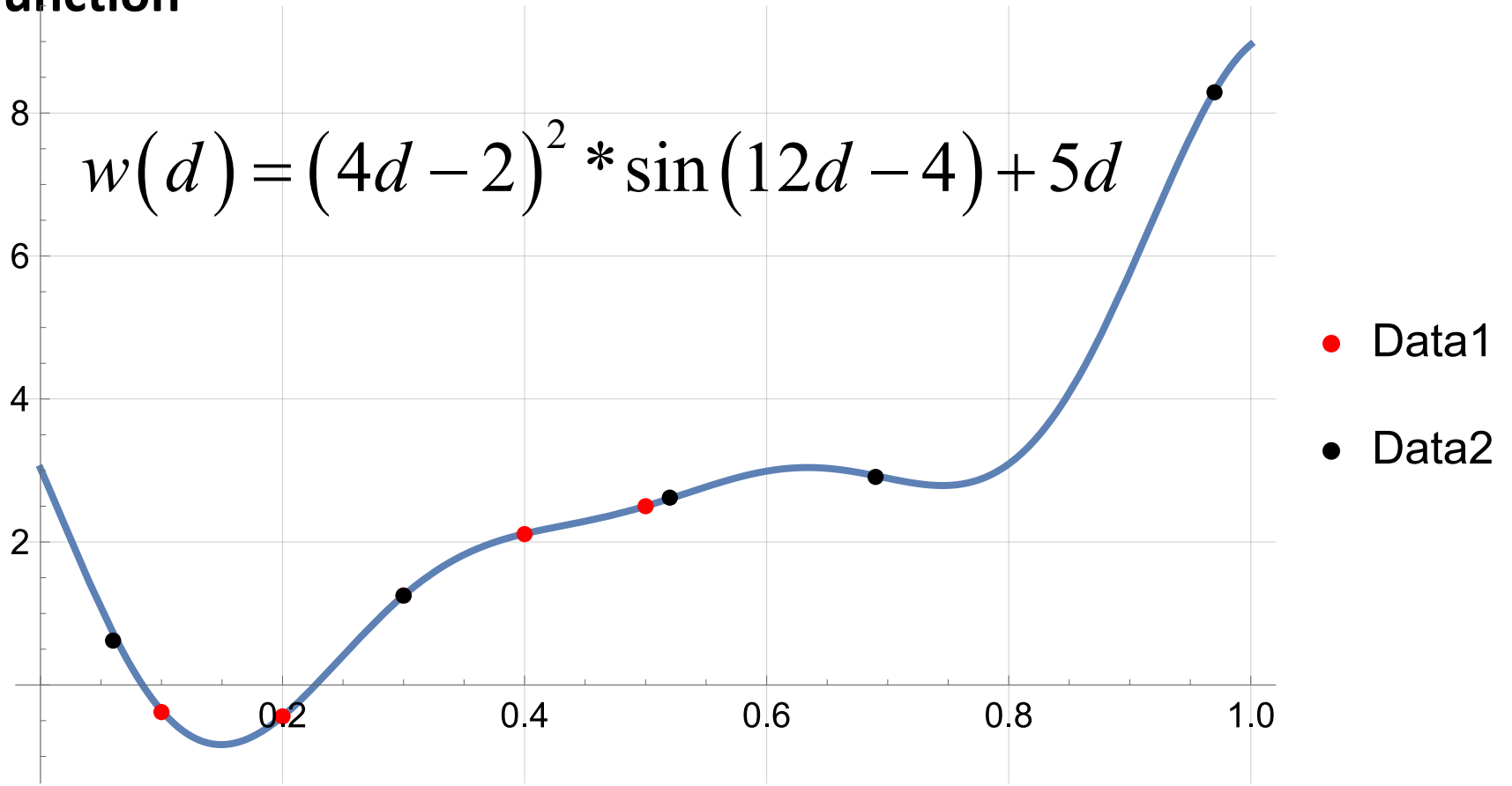
The following quantities are scalars $\mathbf{1}^T \hat{\mathbf{R}}^{-1}\hat{\mathbf{r}}(d)$ and $\mathbf{1}^T \hat{\mathbf{R}}^{-1}\mathbf{1}$

Examples

One factor

We will use **Ordinary Kriging** and Mathematica commands. In Classwork you will use JMP. Both work with a Bayesian scheme

Let us plot the two experiments we discussed and the **true response function**



One factor

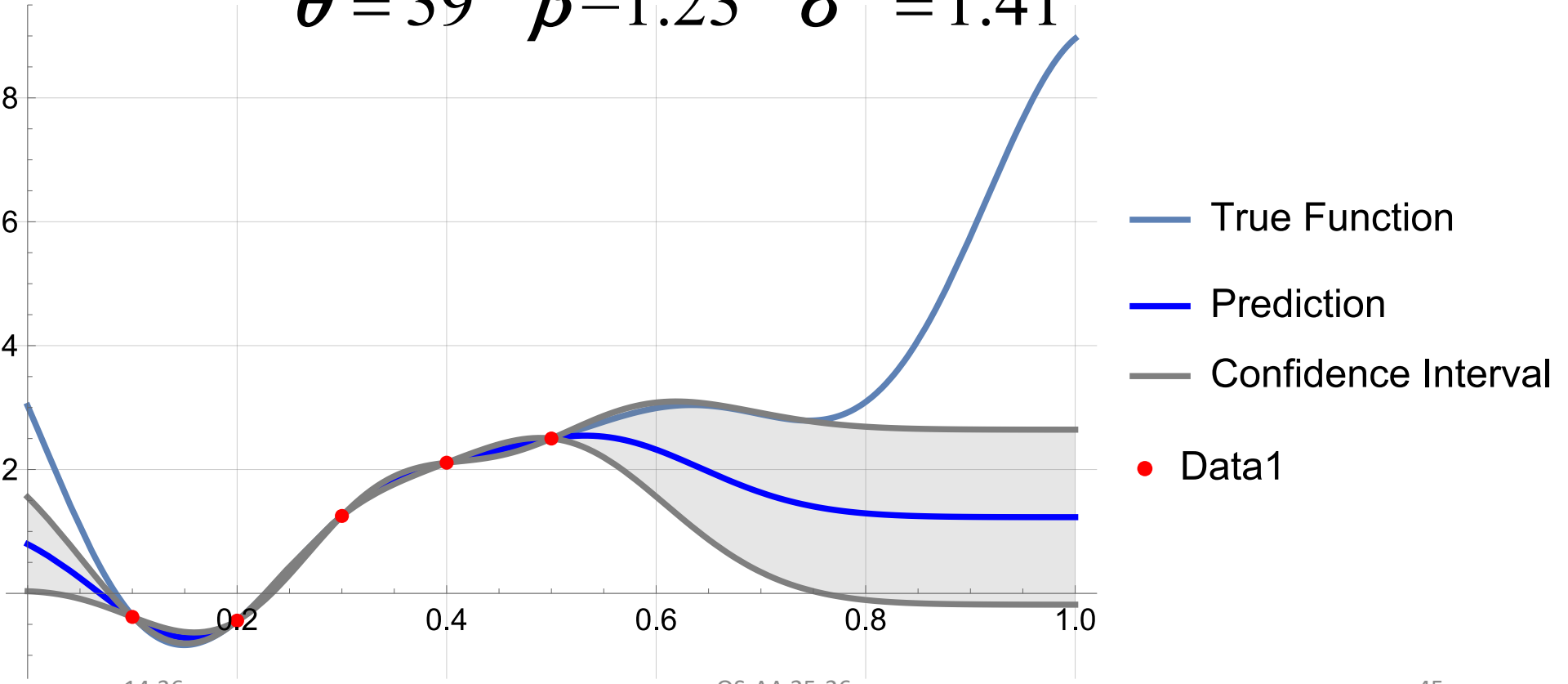
We will use **Ordinary Kriging** and Mathematica commands and p=2. In Classwork you will use JMP

Let us see the two experiments we discussed

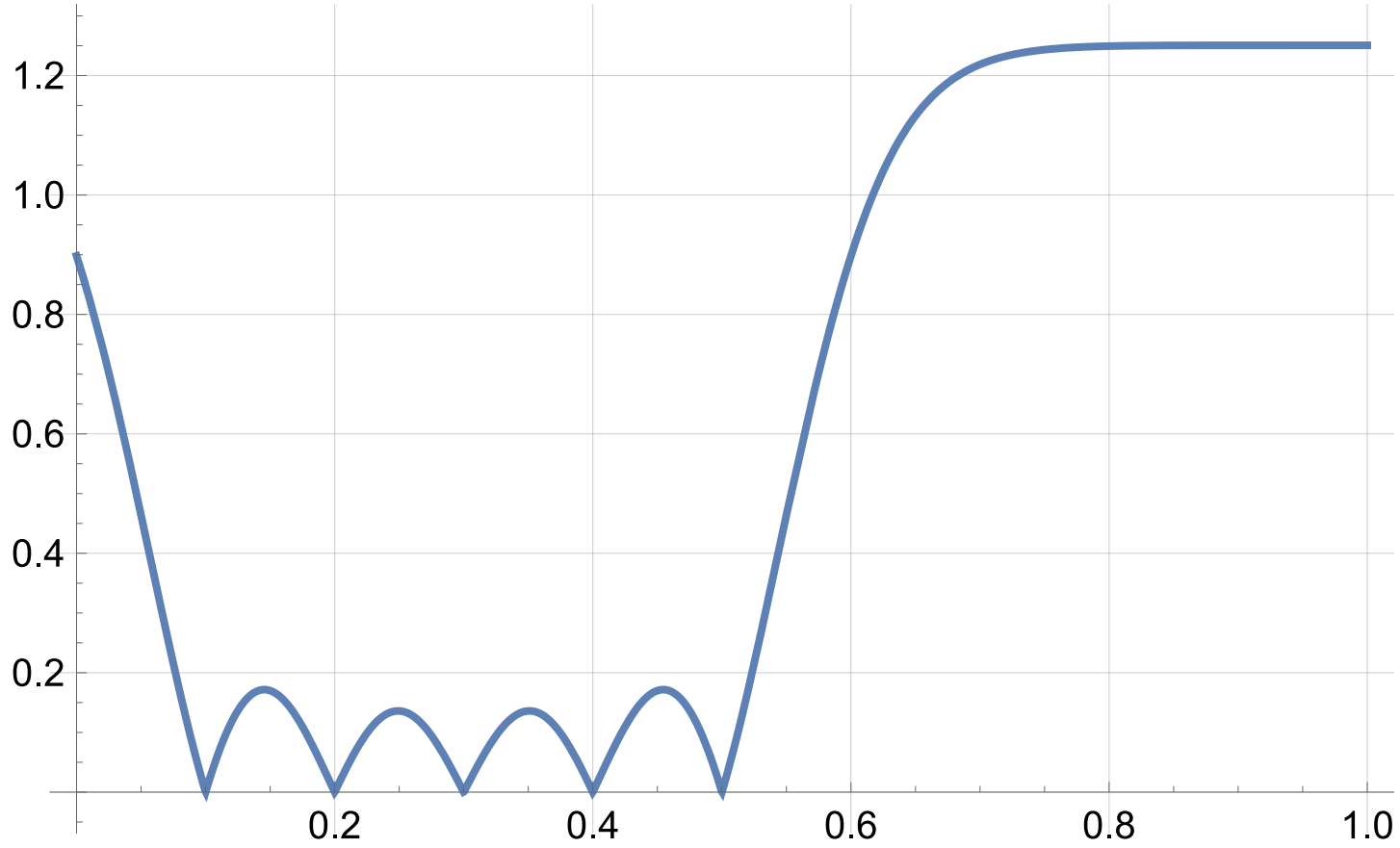
First Experiment

{(0.1,-0.38),(0.2,-0.44),(0.3,1.25),(0.4,2.11),(0.5,2.50)}

$$\hat{\theta} = 39 \quad \hat{\beta} = 1.23 \quad \hat{\sigma}^2 = 1.41$$



Standard Deviation of the Prediction (from Mathematica)



JMP Output (Analyze-> Specialized Model-> Gaussian Process)

Model Report

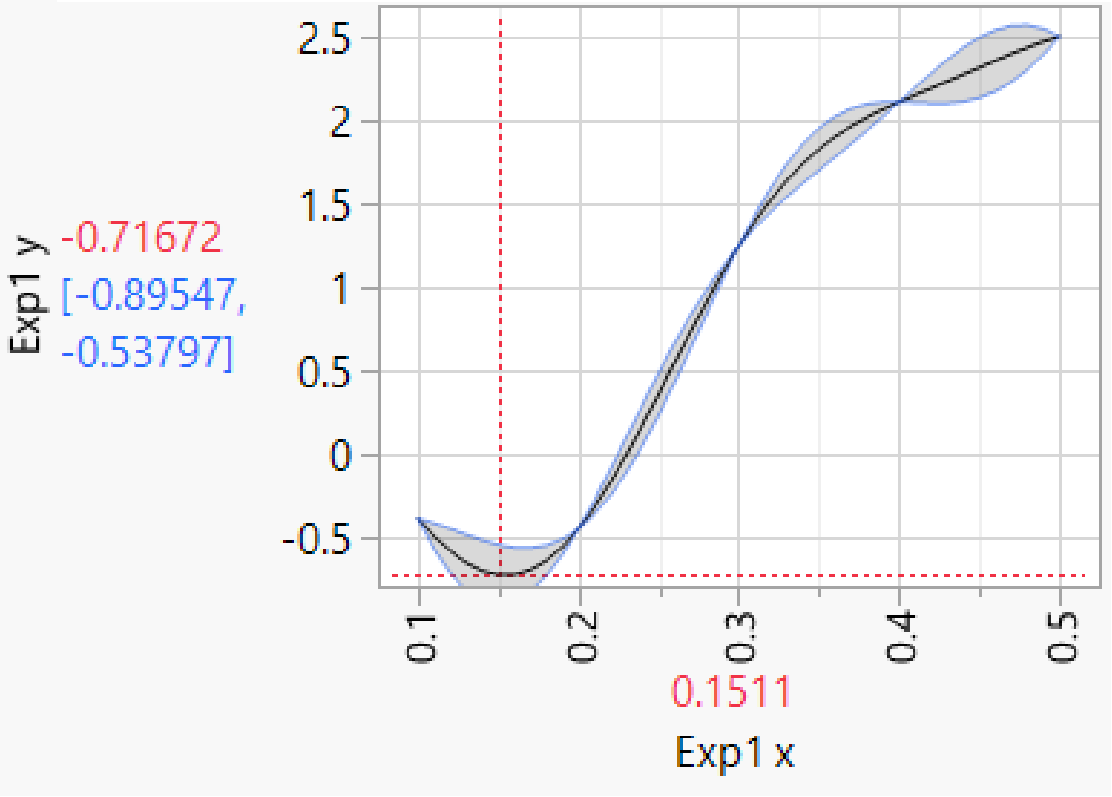
Column	Theta	Total Sensitivity	Main Effect	Exp1 x Interaction
Exp1 x	38.989354	1	1	.

μ	σ²
1.2307384	1.4095862

-2*LogLikelihood
12.496253

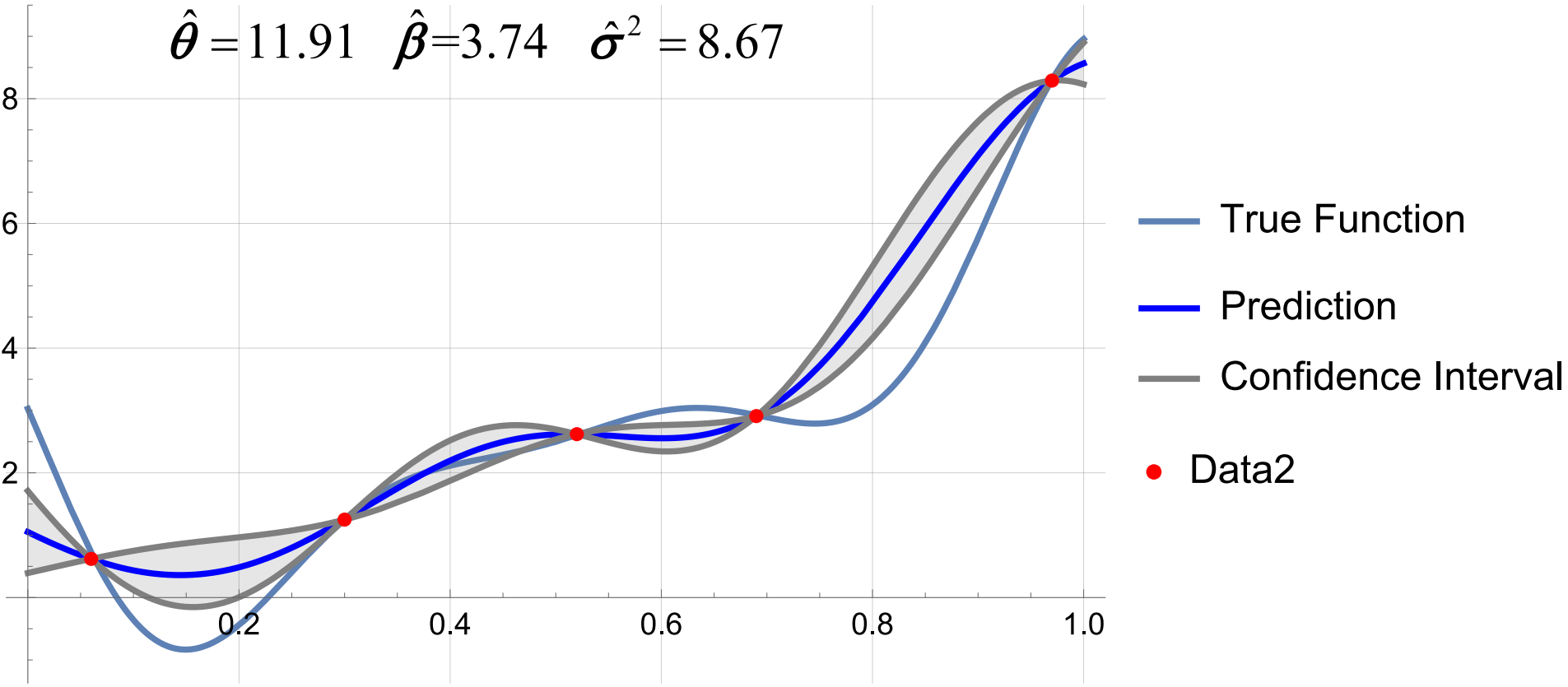
Fit using the Gaussian correlation function.

$\hat{\theta} = 39 \quad \hat{\beta} = 1.23 \quad \hat{\sigma}^2 = 1.41$



Second experiment

{(0.06,0.62),(0.3,1.25),(0.52,2.62),(0.69,2.91),(0.97,8.29)}



A great improvement!!

JMP Output

Model Report

Column	Theta	Total Sensitivity	Main Effect	Exp2 x Interaction
Exp2 x	11.914451	1	1	.

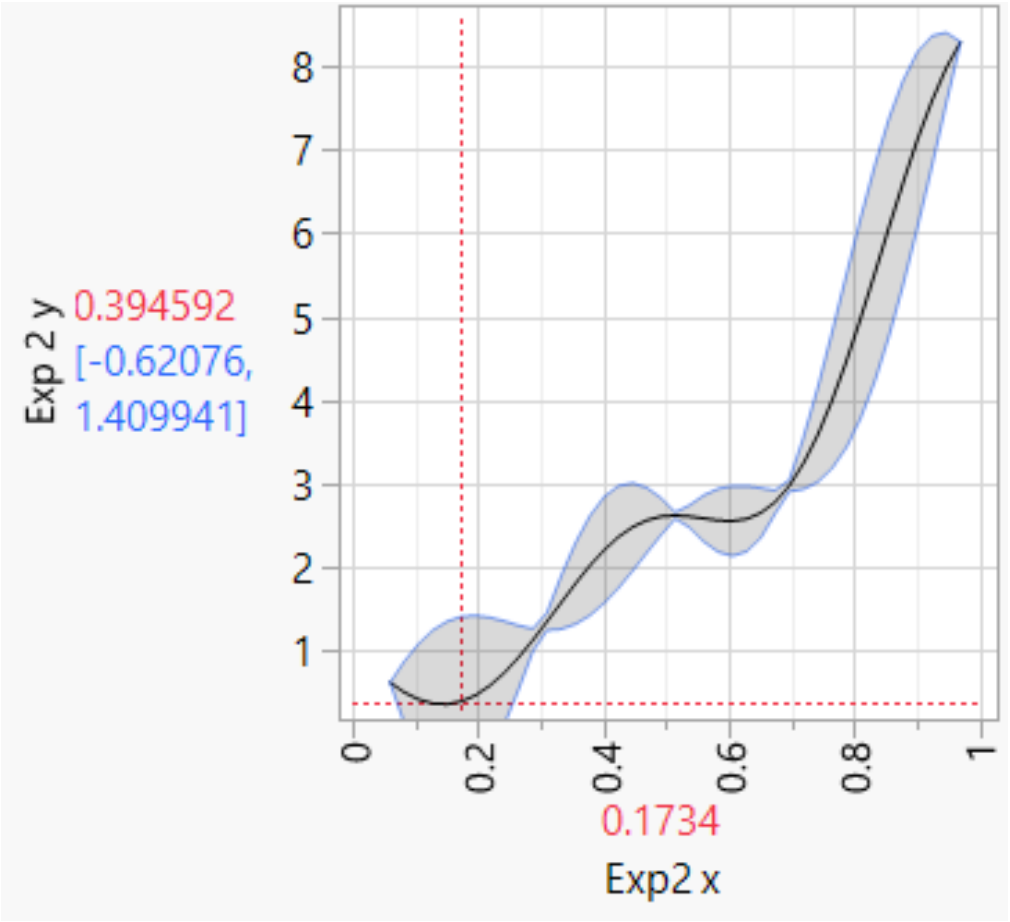
μ	σ²
3.7392622	8.6645673

-2*LogLikelihood
23.02291

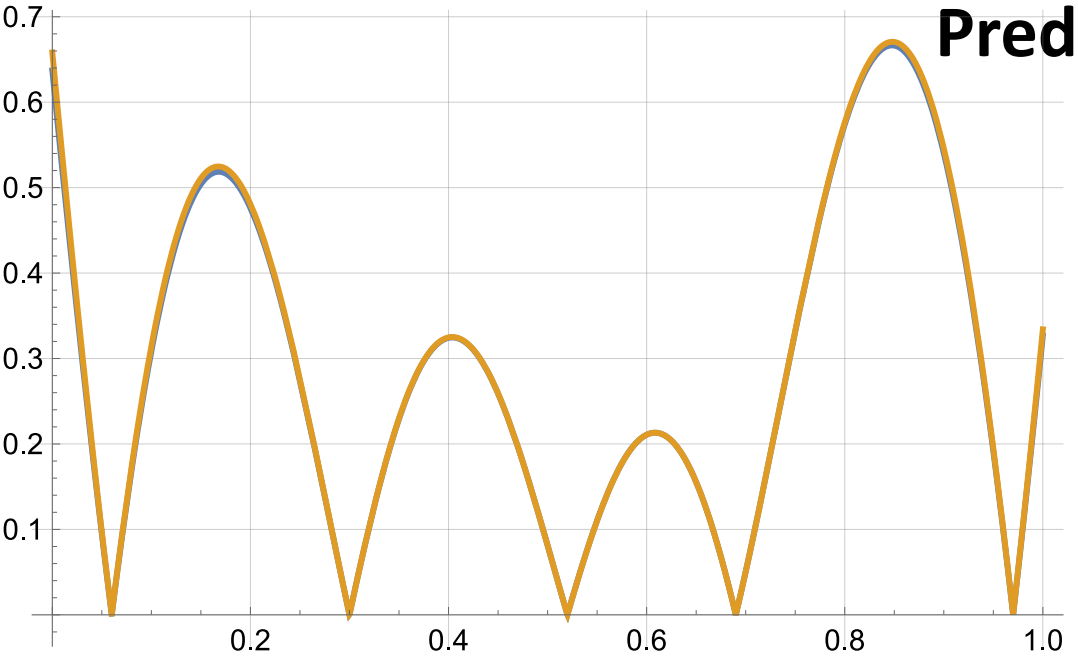
Fit using the Gaussian correlation function.

Mathematica output

$$\hat{\theta} = 11.91 \quad \hat{\beta} = 3.74 \quad \hat{\sigma}^2 = 8.67$$

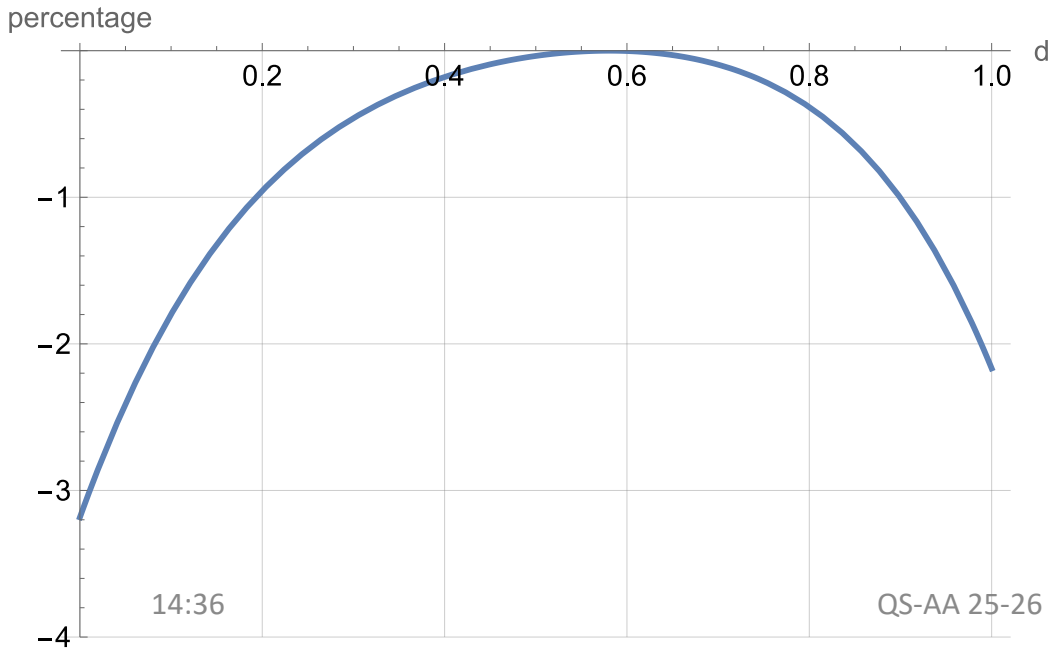


Prediction Standard Deviation



stdDevSimpl(11.9, (3.74021)[[1, 1]], *d*)

stdDevPredError(11.9, (3.74021)[[1, 1]], *d*)



Percentage difference:
(simplified-correct)/correct*100

Expansion: Multifactor Gaussian Process

$$d \to \mathbf{d} \quad \text{with dimensions } (q*1)$$

$$\left\{\mathbf{R}\left(\mathbf{d}_i,\mathbf{d}_j\right)\right\}_{ij} = \prod_{h=1}^{q} R_h\left(d_{ih},d_{jh}\right) = \prod_{h=1}^{q} \exp\left(-\boldsymbol{\theta}_k \left|d_{ih}-d_{jh}\right|^{p_h}\right)$$

Weights apply to each treatment dimension

$$\left\{\mathbf{r}\left(\mathbf{d},\mathbf{d}_j\right)\right\}_{j} = \prod_{h=1}^{q} r_h\left(d_h,d_{jh}\right) = \prod_{h=1}^{q} \exp\left(-\boldsymbol{\theta}_k \left|d_{ih}-d_{jh}\right|^{p_h}\right)$$

Usually, to reduce the computational burden, we set

$$p_k = p = 2 \qquad \forall k = 1,..,q$$

Formulas are the same

Here is a nice tool to play with

https://distill.pub/2019/visual-exploration-gaussian-processes/